Tiny Frames

E-COMMERCE FULL-STACK PROJECT

Programme: Full-stack Development

Major: Web Development

Karolina A. Borucka

kabo0004@stud.ek.dk

Concept & Aesthetic

The opportunity

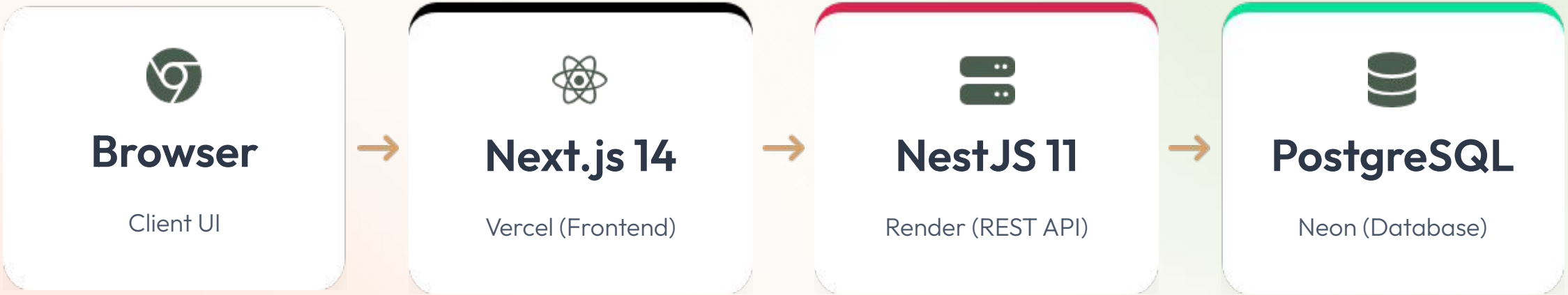
Finding calm, highly curated Scandinavian-style art prints and stickers for kids' rooms is difficult on generic, cluttered marketplaces.

- Demand for a soft, curated gallery experience.
- Need for a simple, distraction-free checkout flow.
- Emphasis on soft colors, clear typography, and "art for little walls".

The solution: Artshop

A custom-built B2C e-commerce platform demonstrating end-to-end full-stack capabilities, complete with robust user accounts, cart management, and a secure admin dashboard.

System Architecture



Strict separation of concerns: The frontend never connects directly to the database.

All data flows securely through the REST API layer.

Technology Stack

Frontend

- Next.js 14 (App Router)
- React 18 & TypeScript
- TanStack React Query
- Context API (Auth/Cart)
- Custom CSS & Scandi Tokens

Backend

- NestJS 11 & TypeScript
- Prisma ORM
- JWT (HTTP-only cookies)
- class-validator DTOs
- Swagger API Docs

Infra & QA

- Vercel, Render, Neon
- GitHub Actions CI/CD
- Cypress 15 (E2E)
- Jest (Unit Testing)
- Sentry Error Monitoring

Database Design & ORM

Core Entities (Prisma)

- **User:** Roles (CUSTOMER, ADMIN)
- **Product & Category:** Inventory logic
- **Order & OrderItem:** Price snapshots
- **Cart, Review, Favorite**

```
// Checkout requires atomic transaction
prisma.$transaction([
  decrementStock(),
  createOrder(),
  clearCart()
])
```

Raw SQL Objects

Beyond the ORM, a raw migration handles advanced database requirements:

View	vw_product_ratings
View	vw_customer_order_totals
Function	fn_product_stock()
Trigger	product_set_updated_at

Backend REST API

Architecture Paradigm

Built with **NestJS** utilizing decorators and dependency injection for a highly scalable structure.

- Data validation using DTOs and ValidationPipe.
- Auto-generated docs at /api/docs.
- Secured via JwtAuthGuard & RolesGuard.

Example Endpoints

GET /products?categoryId=...

POST /auth/login

POST /orders/guest

PATCH /cart/items/:id

DELETE /admin/products/:id

Frontend routing & state

Next.js App Router

29+ robust pages mapping B2C journeys and Admin tooling. Protected routes utilize client-side layout redirects combined with strict API-level enforcement.

- /products (URL queries for filters)
- /checkout (Guest or Authenticated)

State Management

React Context: Handles cross-cutting client state globally (e.g., AuthProvider, CartProvider).

TanStack React Query: Manages server state elegantly, handling loading flags and caching (e.g., staleTime: 60s for products) to reduce API load.

Admin dashboard

Integrated directly into the site chrome for a unified brand experience. Protected by the ADMIN DB role.

Features: product CRUD, order tracking, customer management.

Security Implementations

HTTP-Only cookies

JWTs are stored securely, mitigating XSS token theft risks compared to localStorage.

Password hashing

Passwords are never stored in plain text, utilizing **bcrypt** prior to database insertion.

CORS policies

Backend strictly whitelists frontend origins via environment variables (FRONTEND_URL).

SQL injection

Prisma ORM automatically parameterizes queries, effectively preventing SQL injection attacks.

Quality Assurance & Testing

Unit Testing (Jest)

Focuses strictly on critical backend business logic and service layers to ensure data integrity.

- ProductsService test suite
- OrdersService test suite
- Verifies database transaction rollbacks and complex checkout mathematics.

E2E Testing (Cypress)

7 comprehensive E2E tests recorded securely to Cypress Cloud (Project ID: nj41qj).

- **smoke.cy.ts:** Basic navigation & custom 404s.
- **shop.cy.ts:** Searching, filtering, product view.
- **auth.cy.ts:** Custom cy.session login, full cart flow.

CI/CD & Isolated Testing Environments

1. GitHub Actions Trigger

Push or PR to `main` branch initiates automated workflows.

2. Build & Test Pipeline (ci.yml)

Runs npm ci, executes Jest unit tests, and builds Next.js/NestJS frameworks.

3. E2E pipeline (cypress.yml) & the CI DB strategy

Instead of connecting to the production Neon DB, the CI pipeline spins up a **temporary, isolated PostgreSQL database**. It runs migrations and seeds data specifically for the test run. This guarantees tests are fast, predictable, and never mutate live production data.

Cloud Deployment Pipeline

Vercel

Next.js Frontend
Serverless Edge
Delivery

Render

NestJS API
Web Service & REST Endpoints

Neon

PostgreSQL
Serverless
Database

Challenges & Future Iterations

Known limitations

Render cold starts: The free tier spins down after inactivity, causing a ~30s delay on the initial API fetch.

Image storage: Render disk storage is ephemeral. Currently relying on remote placeholder URLs; physical uploads wipe on server restart.

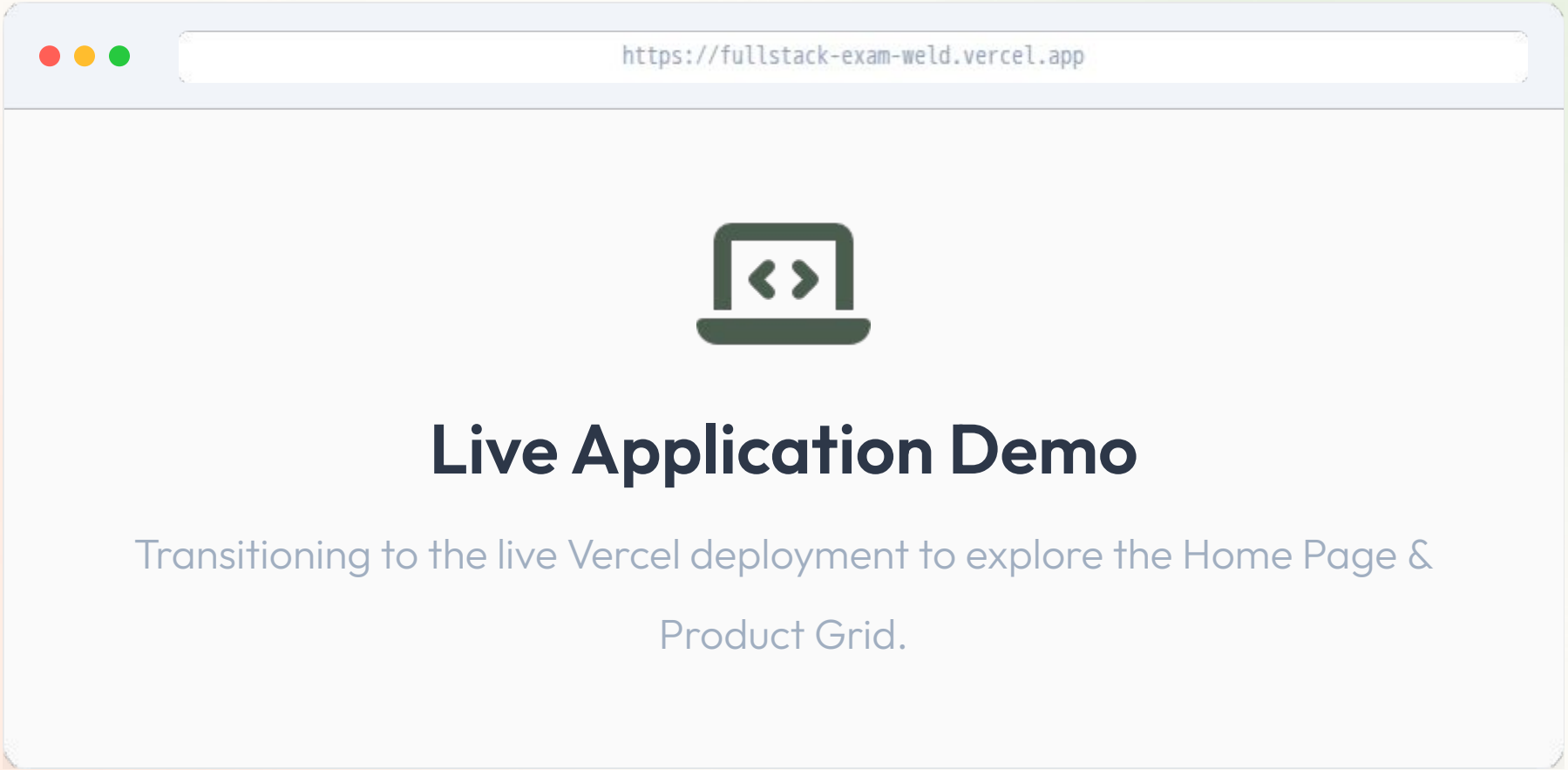
Next steps

Integrate an S3 bucket or Cloudinary for persistent image CDN delivery.

Implement real payment gateways (e.g., Stripe webhooks) replacing the simulated checkout.

Expand the Artist profile schema for multi-vendor support.

Website presentation



Thank You.

Any questions?

LINKS

- github.com/karo4377/fullstack-exam
- fullstack-exam-weld.vercel.app
- /api/docs (Swagger)